

Zadanie indywidualne

Prosta sieć neuronowa dla zbioru Iris w TensorFlow

Materiał do samodzielnej realizacji przez kursantów

Temat	Klasyfikacja wieloklasowa na zbiorze Iris
Technologia	Python, TensorFlow / Keras, scikit-learn
Forma pracy	Indywidualna
Rezultat	Plik .ipynb albo .py z kodem, wykresami i wnioskami

1. Cel zadania

Celem zadania jest zbudowanie prostej sztucznej sieci neuronowej w bibliotece TensorFlow / Keras, która będzie klasyfikować gatunki irysów na podstawie cech liczbowych kwiatów. **Zadanie prowadzi kursanta przez pełny proces: dane, przygotowanie, model, trening, predykcję, analizę błędów i wnioski.**

Wykorzystany zostanie klasyczny zbiór danych Iris, zawierający trzy gatunki irysów:

- Iris setosa
- Iris versicolor
- Iris virginica

Każda próbka opisana jest czterema cechami:

Cecha	Opis
sepal length	długość działki kielicha
sepal width	szerokość działki kielicha
petal length	długość płatk
petal width	szerokość płatk

2. Biblioteki

W projekcie użyj następujących bibliotek:

```
tensorflow
keras
numpy
pandas
sklearn
matplotlib
```

3. Część 1 - Wczytanie danych

Wczytaj zbiór danych Iris z biblioteki sklearn.datasets. Następnie wykonaj następujące kroki:

1. Wyświetl podstawowe informacje o zbiorze.
2. Sprawdź liczbę próbek.
3. Sprawdź liczbę cech wejściowych.
4. Sprawdź dostępne klasy.

5. Utwórz obiekty X oraz y.

X - dane wejściowe

y - etykiety klas

4. Część 2 - Przygotowanie danych

Podziel dane na zbiór treningowy i testowy. Przykładowy podział:

80% danych treningowych

20% danych testowych

Następnie wykonaj standaryzację danych wejściowych za pomocą StandardScaler.

Ważna zasada: scaler dopasowujemy tylko na danych treningowych. Dane testowe tylko transformujemy. Dzięki temu nie dochodzi do wycieku informacji ze zbioru testowego do treningu.

5. Część 3 - Budowa modelu sieci neuronowej

Zbuduj prostą sieć neuronową w TensorFlow / Keras. Model powinien zawierać:

6. Warstwę wejściową dla 4 cech.
7. Co najmniej jedną warstwę ukrytą Dense.
8. Funkcję aktywacji relu w warstwie ukrytej.
9. Warstwę wyjściową z 3 neuronami.
10. Funkcję aktywacji softmax na wyjściu.

Przykładowa struktura modelu:

Input: 4 cechy

Dense: 16 neuronów, relu

Dense: 8 neuronów, relu

Dense: 3 neurony, softmax

6. Część 4 - Kompilacja modelu

Skompiluj model, używając następujących ustawień:

```
optimizer="adam"  
loss="sparse_categorical_crossentropy"  
metrics=["accuracy"]
```

W komentarzu do kodu wyjaśnij, dlaczego używamy sparse_categorical_crossentropy.

Podpowiedź: etykiety klas są zapisane jako liczby całkowite: 0, 1, 2.

7. Część 5 - Trenowanie modelu

Wytrenuj model przez określoną liczbę epok, na przykład:

```
epochs=50
```

Podczas treningu użyj części danych treningowych jako zbioru walidacyjnego:

```
validation_split=0.2
```

Zapisz historię uczenia do zmiennej history.

8. Część 6 - Ocena modelu

Oceń model na danych testowych. Wyświetl:

- test loss
- test accuracy

Następnie odpowiedz krótko na pytania:

11. Czy model osiągnął wysoką dokładność?
12. Czy wynik jest lepszy niż losowe zgadywanie?
13. Czy funkcja straty jest niska?
14. Czy model prawdopodobnie nauczył się sensownej zależności?

9. Część 7 - Predykcja

Wykonaj predykcję dla danych testowych. Następnie:

15. Oblicz prawdopodobieństwa klas.
16. Użyj funkcji `argmax`, aby wybrać klasę o najwyższym prawdopodobieństwie.
17. Porównaj predykcje modelu z prawdziwymi etykietami.

Przykład:

```
y_proba = model.predict(X_test)
y_pred = np.argmax(y_proba, axis=1)
```

Wyświetl pierwsze 10 wyników w formie:

```
prawdziwa klasa | przewidziana klasa
```

10. Część 8 - Analiza wyników predykcji

Przygotuj analizę jakości modelu. Wykonaj:

18. Macierz pomyłek.
19. Raport klasyfikacyjny.
20. Sprawdzenie, które klasy model rozpoznaje najlepiej.
21. Sprawdzenie, które klasy najczęściej się mylą.

Użyj narzędzi z biblioteki `sklearn.metrics`:

```
confusion_matrix
classification_report
```

11. Część 9 - Wizualizacja procesu uczenia

Narysuj dwa wykresy:

22. Wykres funkcji straty: `loss` oraz `validation loss`.
23. Wykres dokładności: `accuracy` oraz `validation accuracy`.

Na podstawie wykresów odpowiedz:

24. Czy model konverguje?
25. Czy widać oznaki przeuczenia?
26. Czy warto byłoby zwiększyć lub zmniejszyć liczbę epok?
27. Czy model uczy się stabilnie?

12. Część 10 - Eksperyment

Zmień architekturę modelu i sprawdź, jak zmieniają się wyniki. Przetestuj przynajmniej dwa warianty:

Wariant	Architektura	Co porównać
A	Dense(8, relu) Dense(3, softmax)	accuracy, loss, stabilność uczenia
B	Dense(32, relu) Dense(16, relu) Dense(3, softmax)	accuracy, loss, czas uczenia, stabilność uczenia

Na końcu odpowiedz:

Który model był najlepszy i dlaczego?

13. Wymagania końcowe

Oddaj plik .ipynb albo .py, który zawiera:

28. Wczytanie danych.
29. Przygotowanie danych.
30. Budowę modelu.
31. Kompilację modelu.
32. Trenowanie modelu.
33. Ocenę modelu.
34. Predykcję.
35. Analizę wyników.
36. Wykresy uczenia.
37. Krótkie wnioski końcowe.

14. Pytania kontrolne

38. Co oznacza warstwa Dense?
39. Dlaczego ostatnia warstwa ma 3 neurony?
40. Do czego służy funkcja softmax?
41. Dlaczego używamy sparse_categorical_crossentropy?
42. Co robi optimizer Adam?
43. Co oznacza accuracy?
44. Co oznacza loss?
45. Co robi funkcja argmax?
46. Co pokazuje macierz pomyłek?
47. Czy model rzeczywiście rozumie kwiaty, czy tylko rozpoznaje zależności w danych?

15. Dodatkowe wyzwanie dla chętnych

Dodaj własny przykład kwiatu:

```
sample = [[5.1, 3.5, 1.4, 0.2]]
```

Przeskaluj go tym samym scalerem, którego użyto wcześniej, wykonaj predykcję i wyświetl:

- przewidzianą klasę
- nazwę gatunku

- prawdopodobieństwa dla wszystkich klas

16. Kryteria oceny

Element	Punkty
Poprawne wczytanie danych	1
Poprawny podział train/test	1
Standaryzacja danych	1
Poprawna budowa modelu	2
Kompilacja i trening	2
Ocena modelu	2
Predykcja i użycie argmax	2
Macierz pomyłek i raport klasyfikacyjny	2
Wykresy loss/accuracy	2
Wnioski końcowe	2
Eksperyment z architekturą	3
Razem	20

17. Myśl końcowa

W tym zadaniu nie chodzi tylko o to, żeby model osiągnął wysoką dokładność. Chodzi o zrozumienie całego procesu:

dane → przygotowanie → model → uczenie → predykcja → analiza błędów

Dopiero wtedy sieć neuronowa przestaje być czarną skrzynką, a zaczyna być narzędziem, które można świadomie budować, kontrolować i interpretować.